International Institute of Information Technology
Bhubaneswar

GeoViz

Computational Geometry Visualizer and Spatial Security Console

Object-Oriented Programming Project Report

SUBMITTED BY

Satyam Dhal

Student ID: b425050

Branch: Computer Science and Engineering (B)

SUBMITTED TO

Dr. Srichandan Shobhanayak

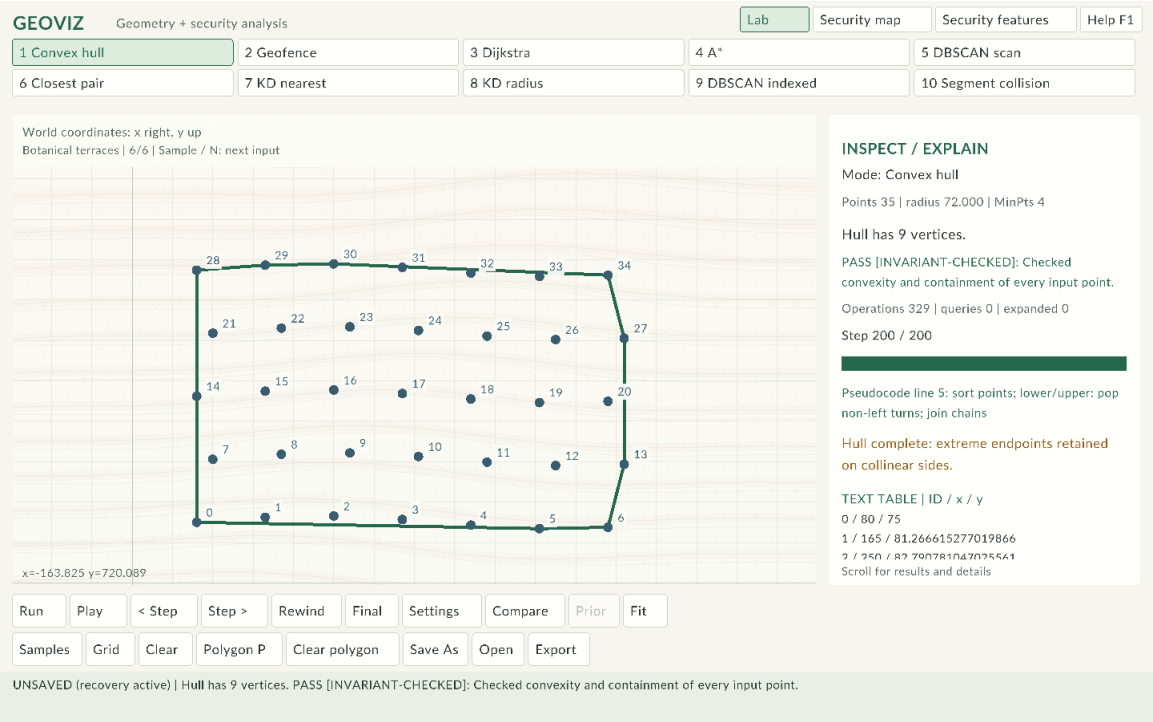
Department of Computer Science and Engineering

25 October 2026

Computational geometry analysis

C++17 desktop application with raylib and an independent computational engine

GeoViz records algorithm decisions so a result can be played, paused, stepped and reproduced. The same engine supports an editable geometry lab and a spatial security console with geographic and behavioural views.



Deliverable	Scope
Application	10 algorithm modes; 60 deterministic editable scenes; 9 security scenarios
Source	Encapsulated domain models, polymorphic algorithms and indexes, tests and build scripts
Documentation	Focused Markdown guides, interactive engine recordings and this print report

Run and review

Open **GeoViz.exe** from the repository root. Select a mode, choose a named input with **Samples / F2**, then **Run** and **Play**. N cycles six inputs per mode, including tidal archipelago and terraced botanical families. Keep the executable beside assets/ and data/. The application works offline and requires no account or installation.

Open **docs/index.html** for interactive recordings. The source guides separately cover algorithms, OOP/C++, security, controls, sample inputs and verification.

From input to checked run

Editing, execution, recorded playback and reproducible outputs

Execution contract

Validate -> freeze scene -> calculate and record -> verify -> inspect/export. Algorithm answers do not depend on animation timing. Editing an input invalidates its current trace. A retained Run owns an independent input/result snapshot for historical comparison.

Action	Control
Run / play-pause	Enter / Space
Step / first-final frame	Left-Right / Home-End
Named sample / parameters	F2 or Samples; N cycles / Settings or O
Add, select, move / query	Left-click, drag / right-click
Pan, zoom, fit	Middle-drag, wheel, F
Polygon / graph links	P for boundary vertices / Shift-click two points
Route endpoints / block link	Pick source-destination or R-T / click edge then B
Named sessions / edit history	Ctrl+S, Ctrl+O / Ctrl+Z, Ctrl+Y

Persistence and separation

SceneIO stores versioned plaintext scenes using full-precision doubles. Polygon drafts reload exactly; a geofence run requires a valid completed boundary. A checked-run report retains its adjacent scene. Security exports preserve both views, source events, settings and reviews. Drag an input file into the application to import it.

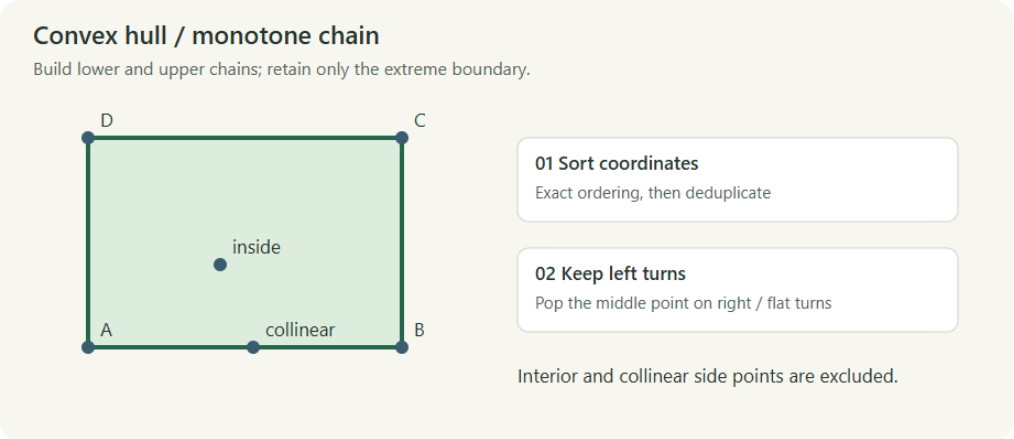
The engine contains no raylib dependency. The presentation layer transforms world coordinates to the screen and reads recorded Frames. Playback changes a cursor; it never recomputes an answer or changes work counters.

Build

On the tested Windows toolchain, run **build.ps1** to compile and execute tests, then build the application. CompilerPath and RaylibRoot can be configured. An engine-only CMake build uses **GEOVIZ_BUILD_APP=OFF**. Detailed commands are in docs/USER_GUIDE.md.

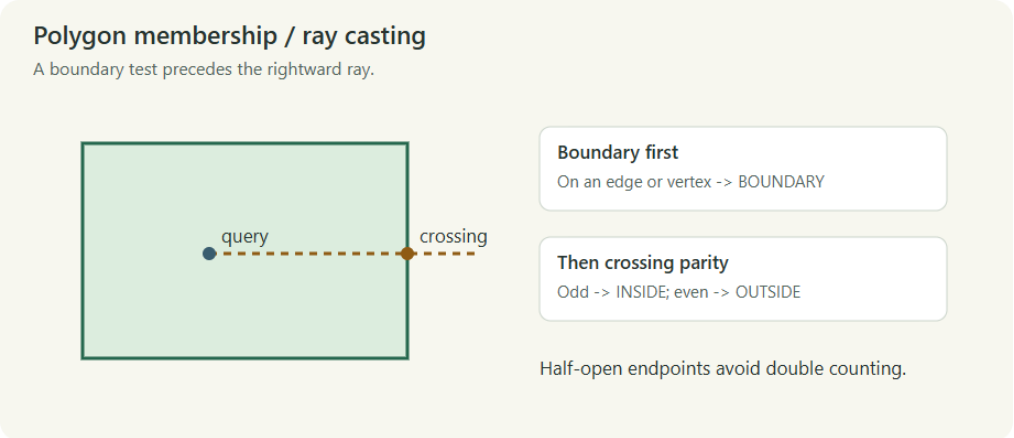
Boundaries and membership

Monotone-chain convex hull and boundary-first ray casting



Hull: sort exact coordinates and remove duplicates; construct lower and upper chains, popping a middle vertex on a clockwise or collinear turn. Join chains without repeating endpoints. Collinear sides retain only extremes. O(n log n) computation; O(n) auxiliary storage.

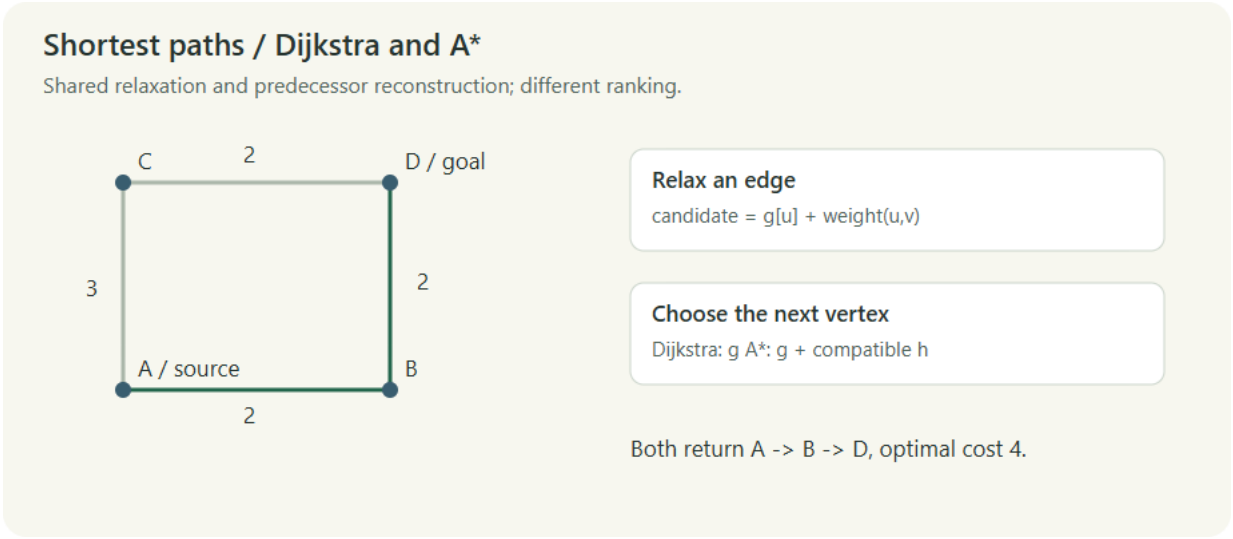
For rectangle corners (0,0), (4,0), (4,3), (0,3) plus (2,1) and (2,0), the hull is the four corners. The trace shows candidates, removals and accepted chains. Verification checks convexity and containment.



Membership: validate a simple polygon; return BOUNDARY if the query lies on an edge. Otherwise toggle parity for rightward ray crossings using a half-open endpoint rule. Query (2,1) is inside the rectangle, (4,1) is boundary, and (5,1) is outside. O(m) per query; polygon validation separately costs O(m squared). A winding-number reference checks the result.

Shortest paths

Dijkstra and A* share one relaxation and reconstruction implementation



Dijkstra: initialize the source to zero, pop the least-cost unsettled vertex, and relax open edges. Strict improvements update tentative cost and predecessor. Stale heap entries are discarded. Settling the goal ends the search; parents reconstruct the route.

A*: rank by f=g+h. Euclidean distance is used on compatible distance graphs; the regular grid uses Manhattan distance divided by its 60-unit spacing. Every open edge is checked against the lower bound. Incompatible weights trigger an explicitly reported h=0 fallback.

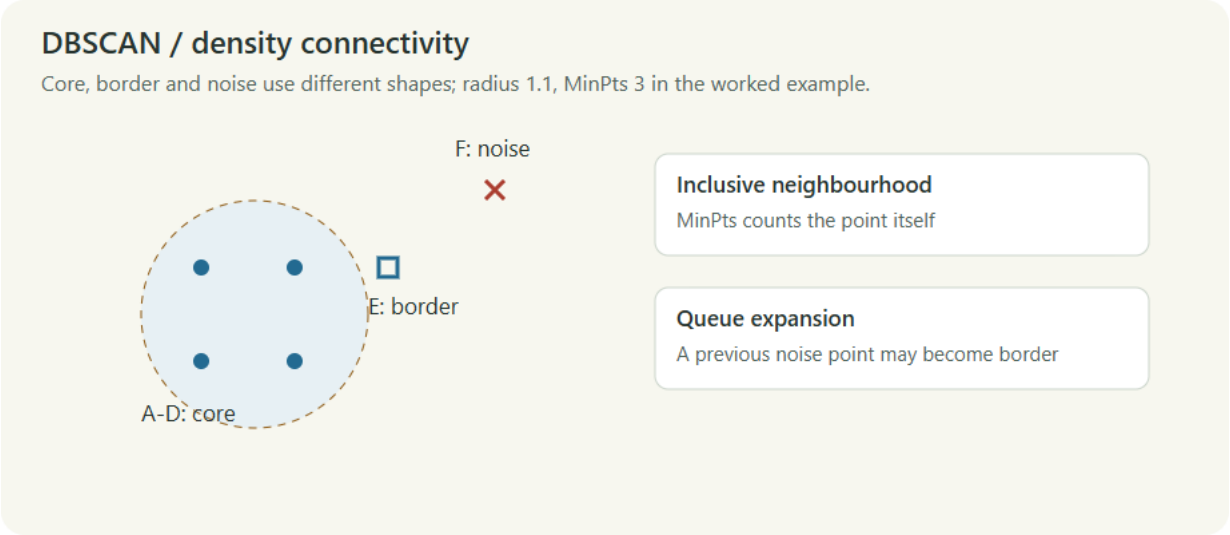
Example	Result
A-B:2, A-C:3, B-D:2, C-D:2	A-B-D has cost 4
Block B-D	A-C-D has cost 5
Block both links to D	NO PATH
Source equals goal	Cost 0

The heap tie order is (f,g,input ID). Costs must be finite and nonnegative. With the binary heap and a consistent enabled heuristic, computation is O((V+E) log V) for the bounded simple graph. A* may settle fewer vertices but is not always faster.

Evidence: frames retain g/f scores, predecessors, frontier and closed vertices. Bellman-Ford checks optimal cost; returned path links and their summed cost are verified separately. Compare uses the identical scene.

Density and spatial indexing

Direct and indexed DBSCAN use one deterministic expansion loop



A **core** point has at least MinPts neighbours within the inclusive radius, counting itself. A non-core neighbour of a core point is **border**. Unreached points are **noise**. A queue expands density-connected core neighbourhoods. Earlier provisional noise can become border; stable input order resolves ambiguous borders.

For A=(0,0), B=(1,0), C=(0,1), D=(1,1), E=(2,1), F=(6,6), radius 1.1 and MinPts 3, A-D are core, E is border and F is noise. Curved swarms and mixed-density gallery inputs demonstrate non-circular groups and parameter sensitivity.

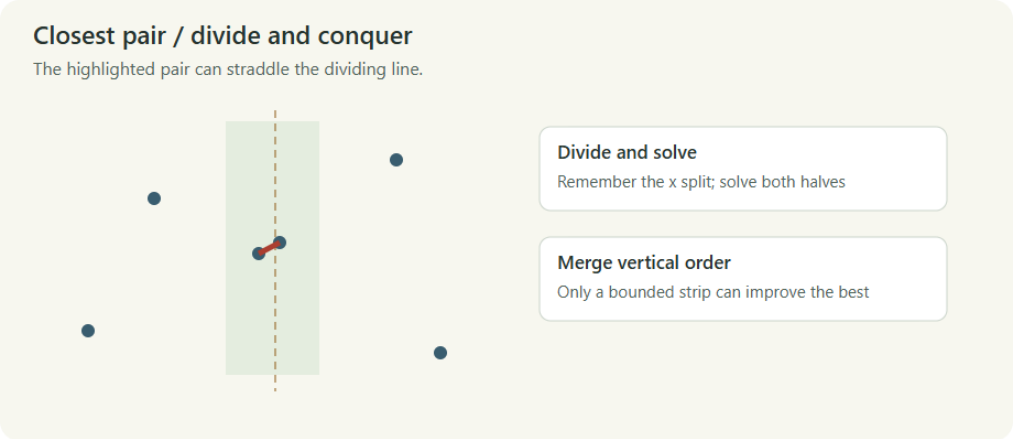
Implementation	Neighbourhood query / cost
Direct scan	LinearIndex tests all points; O(n squared) distance work
Indexed	KDTree prunes boxes outside the radius; expected O(n log n) build; worst-case density work remains O(n squared)

Both strategies return sorted neighbour IDs and use the same expansion code. Frames retain the query circle, queue, labels and point kinds. An independent core-component graph plus border assignment checks the result. Compare reports distance tests, excluding index construction and verification; it is not a timing benchmark.

Density noise is an investigation cue in the security console. It is never sufficient by itself to establish an attack.

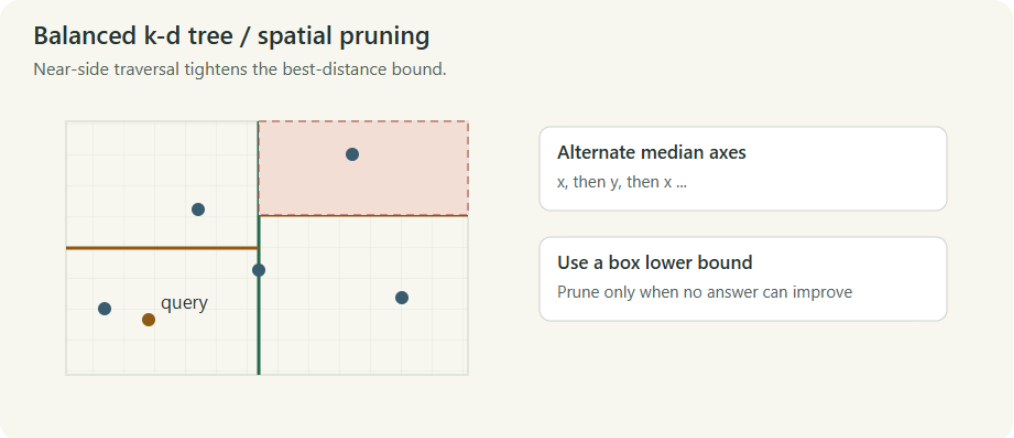
Proximity queries

Closest pair and balanced k-d nearest/radius search



Closest pair: sort by x once; recursively solve halves and merge their y order. Compare candidates in the dividing strip, with at most seven following y-ordered points per candidate. Exact duplicates are handled first and yield zero. O(n log n) computation and O(n) auxiliary storage; exhaustive pair distances verify the answer.

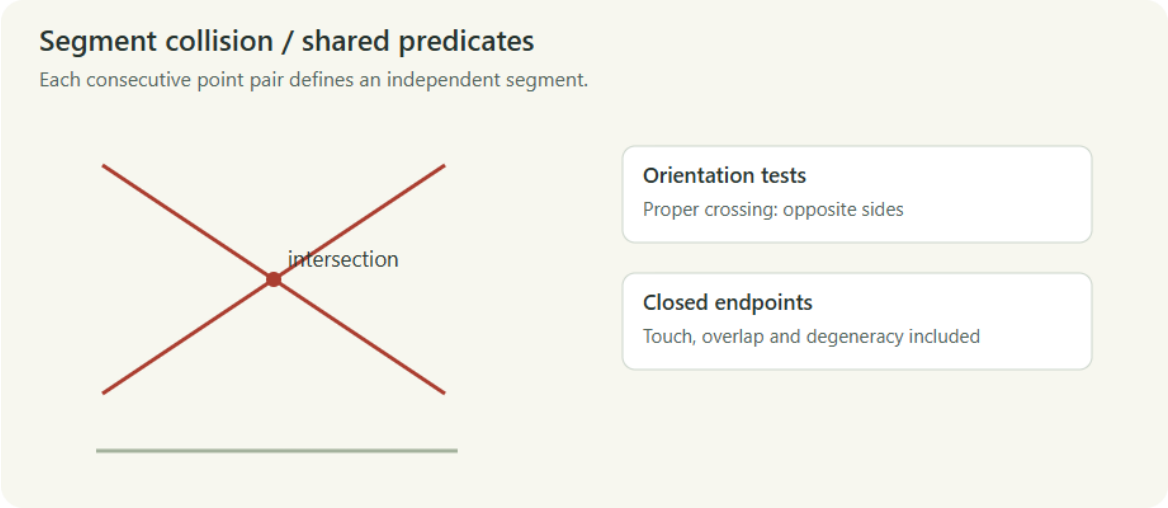
For (0,0), (3,0), (4,4), (4.2,4.1), (8,2), the best distance is sqrt(0.05), approximately 0.223607. Fewer than two points produce no pair.



K-d tree: alternate axes and select medians. Visit the near branch first; the query-to-box distance bounds every point in a region. Prune strictly worse nearest boxes, or boxes outside the radius. Exact radius boundaries are included; nearest ties choose the lowest input ID. Rebuild for every new input snapshot. Expected O(n log n) build; queries can visit O(n) nodes. Radius IDs are sorted. Exhaustive scans verify results.

Shared geometry and bounds

Collision predicates, numerical policy and bounded recordings



Segment collision: consecutive point pairs define closed segments. Test each unordered segment pair once. Opposite orientations detect proper crossings; on-segment tests add touches, overlap and zero-length cases. An unmatched final point is unused. Computation is O(s squared) for s segments.

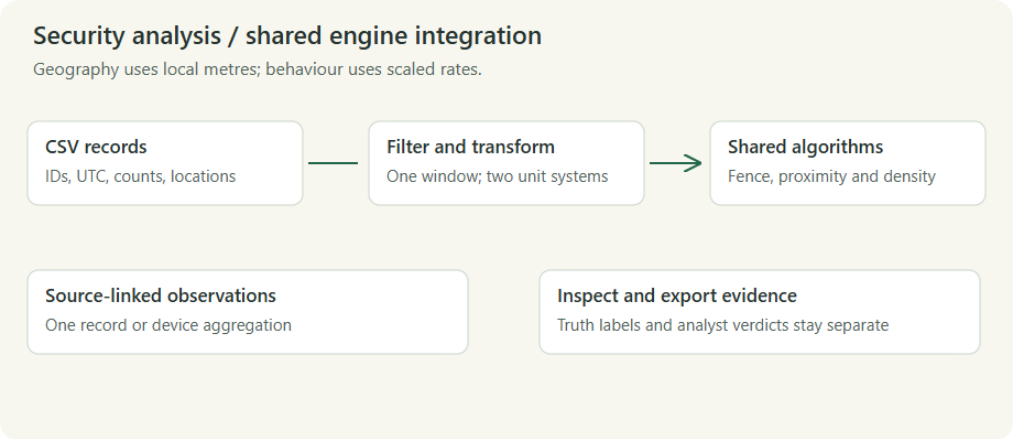
Crossing, endpoint, overlapping, disjoint and degenerate cases have explicit expected tests. No separate runtime collision oracle is claimed. Orientation, distance and membership predicates are reused by the other algorithms.

Policy	Definition
Coordinates	Finite doubles, within +/-10,000,000
Orientation tolerance	1e-9 x max(1, length AB x length AC)
Ordering / radius	Exact strict coordinate sort / inclusive radius without added tolerance
Scene bounds	256 points, 256 polygon vertices, 4,096 edges
Trace bounds	4,096 snapshots; prefix and final state retained, truncation reported

These are floating-point predicates rather than exact arithmetic. Snapshot construction, recording, rendering and independent verification add costs beyond the algorithm bounds. Operation counters are generated by computation and remain unchanged during playback.

Security analysis pipeline

Common filtering; separate geographic and behavioural measurements



Geography: each selected event is projected into local east/north metres. The equirectangular model uses R=6,371,000 m and default origin 20.2961 N, 85.8245 E. Extent is limited to 50 km and origin latitude to +/-80 degrees. The campus layer and event logs are synthetic study data, not a surveyed real college map.

Behaviour: selected records are grouped by device. Divide requests and failures by the complete window duration in minutes, then divide each rate by its explicit scale. Default scales are 10 requests/min and 1 failure/min. For 400 requests and 40 failures over ten minutes, the plotted point is (4,4). Feature distances are not metres.

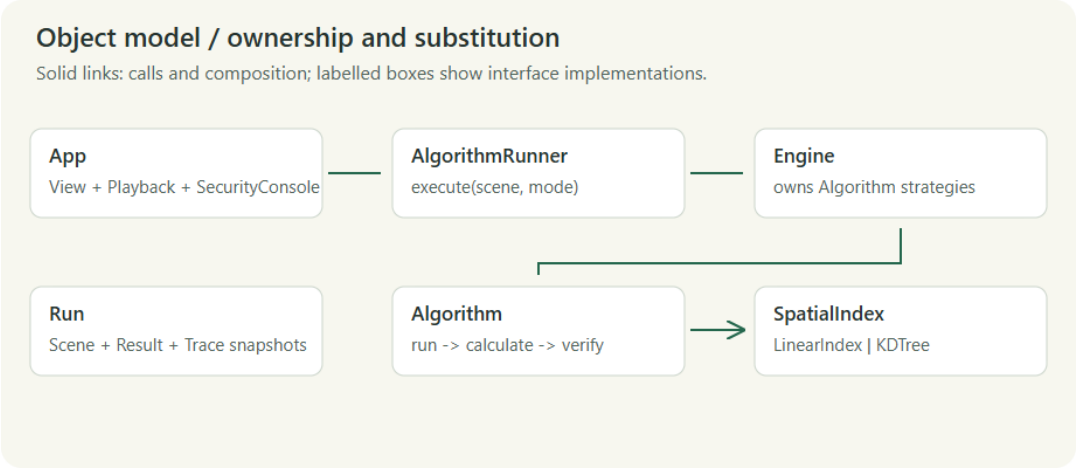
Scenario group	Included datasets
Reference inputs	Compact reference, campus operations, research network
Boundary and behaviour	Perimeter inspection, authentication bursts, device movement
Filtering and density	Low-activity telemetry, time-window boundaries, dense campus traffic

Apply the common [start,end) UTC window and exact user/device filters before transformation. Source indices remain linked to each observation. Query geofences and proximity in the map; run DBSCAN on either view. Settings control units and parameters; switching views preserves the analysis context.

Review rule: feature noise OR strictly outside the permitted polygon. Boundary is permitted. Known sample labels are evaluation truth, never algorithm inputs; analyst verdicts are stored separately. The compact sample has TP=1, FP=1, TN=10, FN=0. The false positive is an approved backup. Export preserves source events, settings, results and reviews for reproduction.

OOP and modern C++

Encapsulation, substitution, composition and resource ownership



Design / feature	Use and significance
Encapsulation	Scene owns topology; validated edits update revisions. Const views prevent untracked mutation.
Template Method	Algorithm::run validates, calls the derived calculate(), and verifies.
Runtime polymorphism	Engine owns Algorithm strategies through unique_ptr; App depends on AlgorithmRunner.
Shared route inheritance	Dijkstra and A* specialize only the heuristic; the search loop stays shared.
Index and adapter Strategies	SpatialIndex selects scan/tree queries; LogAdapter selects event ingestion.
Composition and snapshots	Result contains Stats/Trace; Run freezes input/result; SecurityConsole owns source context.
RAII and ownership	Window, Ui and DrawingSession release resources; owning polymorphic bases have virtual destructors.
Modern C++	Move semantics, const access, enum class, override/final, STL containers/algorithms, lambdas and structured bindings serve concrete model/search tasks.

SceneCanvas renders a const frame context in separate layers. SettingsPanel owns draft fields and validates candidate models before App commits. Tests enforce abstract interfaces, virtual destruction and read-only ownership boundaries.

Recoverable OOP workspaces

Version 2: validated transitions, Memento history and reproducible evidence

Preserve and explain input changes

Lab geometry, mode, view and selections survive security workspace switches. EditHistory uses the Memento pattern for bounded undo/redo, separate from completed Run history. A drag is one edit. Named Windows sessions, recent files, dirty markers and local recovery generations protect investigations during replacement or restart.

Validated state and transactional files

Settings use round-trip precision and contextual UTC fields. Security settings/events are replaced through validated candidates. PreparedSecurity builds both views and treats the campus map as optional before committing an import. AtomicFiles closes and flushes staged streams, then publishes the generation with rollback on ordinary filesystem errors. Recovery backups remain if restoration itself fails; this is not a power-loss journal.

Teaching and analysis

The desktop timeline seeks recorded pseudocode line IDs and route tables. Frozen comparison shows equivalent algorithms side by side; presentation mode hides editing bars. Keyboard focus, text sizing and point tables supplement shape and colour cues. Structured evidence declares parameters, units, source revision, consistency hashes and per-event candidate reasons. Notes and UTC review times remain separate from synthetic truth. Parameter sensitivity is a teaching comparison, not a detection-accuracy claim.

Focused responsibilities

Engine interfaces are split by geometry, scenes, traces, algorithms, persistence and security. CSV parsing, security analysis and snapshot/evidence I/O occupy separate translation units. App has focused implementation sections, while EditHistory, PreparedSecurity and AtomicFiles expose testable graphics-independent responsibilities. Existing Strategy, Template Method, const snapshots and RAII remain central.

The complete 40-item implementation and evidence checklist is docs/REVIEW_RESOLUTION.md. Source builds, authoring locks, CI configurations and release provenance are documented in docs/BUILD_RELEASE.md.

Verification and delivery

Executed checks, implementation coverage and practical limits

Check	Result
GCC 15.2.0 release, C++17, strict warnings as errors	Passed
Engine suite; fixed seed 425050	1766 assertions passed
Focused regressions	Numerics, writes, transitions and structured evidence
GCC/Clang ASan+UBSan	Exact-revision CI artifacts linked from repository
CMake desktop build / isolated CTest	Passed
UI state and rendered scenarios	77 states at 960x640 and 1440x900
Offline guide	24 recordings, controls, navigation and narrow layout verified

Acceptance coverage

Hull containment/convexity, polygon boundary classification, Dijkstra/A* optimal-cost agreement, unreachable routes, density labels, closest-pair/query references, edit invalidation and persistence are checked. Coverage includes indexed DBSCAN, k-d queries, closest pair and both security views. All 60 laboratory scenes and nine security scenarios are included.

Reproducible evidence

docs/SAMPLES.md links editable inputs and checked reports. docs/verification/ contains concise execution logs. docs/VERIFICATION.md maps requirements to implementations and states verification limits. The release manifest records source/data/executable hashes for the delivered folder.

Practical limits

Automated engine, application-state and rendering checks support the documented behaviour; they are not a proof for all inputs and display configurations. The executable targets Windows x64. Floating-point geometry and local projection are approximate. Trace memory is capped. Indexed search can still be linear and density expansion quadratic. Synthetic security evaluation does not establish production accuracy.

The console reads selected local exports and provides no authentication, secure storage, live production account connector or automatic incident response. Review candidates require context. The source and documentation remain editable for repository review.